

Database System(Lab)



Submitted To:

Rana Rashid Mahmood Khan

Submitted By:

2024-CE-25: Asmad Soban
2024-CE-28: Salman Ahmed
2024-CE-36: Haider Arshad
2024-CE-46: Ubaid Ali

Table of Contents

Table of Contents.....	2
1. Introduction	3
2. Problem Statement	3
3. Methodology	3
3.1 Development Stack	3
3.2 Architecture Pattern	4
3.3 Development Process.....	4
4. Project Structure & Implementation.....	4
4.1 Folder Structure.....	4
4.2 Domain Models.....	4
4.3 Business Logic in Service Layer	5
5. Database Design	6
5.1 Tables	6
5.2 Key Constraints & CHECK Constraints.....	6
5.3 Views.....	6
5.4 Stored Procedures.....	7
5.5 Triggers	7
5.6 Transactions.....	7
6. Real-World Application.....	8
6.1 Front Desk Operations.....	8
6.2 Financial Management	8
6.3 Housekeeping Coordination.....	8
6.4 Guest Services	8
6.5 Reporting & Analytics	8
7. Conclusion	9

1. Introduction

The Hotel Booking Management System is a full-stack web application developed as part of the CMPE-232L Database Systems Lab course at UET Lahore. The system is designed to digitize and streamline the core operations of a hotel, including guest management, room booking, employee administration, housekeeping scheduling, payment processing, and invoice generation.

The project follows a structured three-tier architecture: a relational MySQL database at the data layer, an ASP.NET Core MVC application at the business logic and presentation layer, and a browser-based interface for end users. The system was built by Group 011 (CE-4th-A) under the supervision of Sir Rashad Mahmood Khan.

The primary goal of this system is to replace manual, error-prone hotel management processes with a reliable, consistent, and database-driven application that enforces business rules at both the application and database levels.

2. Problem Statement

Traditional hotel management relies heavily on manual processes — paper registers, spreadsheets, and disconnected systems — which lead to a range of operational problems:

- Double bookings due to the absence of real-time room availability checks
- Inconsistent pricing where invoiced amounts do not match the actual room rate
- Untracked cancellations with no automatic refund calculation or booking status update
- No linkage between payments and invoices, causing accounting discrepancies
- Manual housekeeping assignment with no tracking of task progress or completion
- Lack of audit trails for errors, login activity, and financial transactions

This project addresses all of the above by implementing a centralized database-backed system with strict constraints, automated triggers, and connected service layers that keep all data consistent and synchronized.

3. Methodology

The project was developed using an iterative approach, starting with database design and schema definition, followed by backend service implementation, and finally view and controller development.

3.1 Development Stack

Component	Technology
Database	MySQL 8.0
Backend Framework	ASP.NET Core MVC (.NET 8)
Language	C#

ORM / Data Access	Dapper + MySql.Data
Report Generation	Custom PDF Reports
Version Control	GitHub (DBS26CE4thAF011)

3.2 Architecture Pattern

The application follows a layered Repository-Service-Controller pattern:

- Repository Layer — handles all raw SQL queries using Dapper, maps results to domain models
- Service Layer — contains business logic such as auto-calculating TotalAmount, linking cancellations to bookings, and syncing invoice status
- Controller Layer — handles HTTP requests, validates model state, and delegates to services
- View Layer — Razor views for all CRUD operations and reporting

3.3 Development Process

The team first finalized the database schema including all 15 tables, constraints, and relationships. Sample data was inserted to validate the schema. The ASP.NET Core MVC project was then scaffolded with repositories and services generated for each entity. Repositories were then corrected to match actual database column names, and service-layer business logic was added to enforce data consistency.

4. Project Structure & Implementation

4.1 Folder Structure

The ASP.NET Core MVC project is organized under the namespace HotelManagementSystem with the following structure:

- Controllers/ — one controller per entity (BookingController, GuestController, etc.)
- Models/Domain/ — C# domain models mirroring database tables
- Repositories/Interfaces/ — repository interface definitions
- Repositories/Implementations/ — Dapper-based repository implementations
- Services/Interfaces/ — service interface definitions
- Services/Implementations/ — business logic service implementations
- Views/ — Razor .cshtml views organized per controller
- Data/ — DbConnection helper class for MySQL connections

4.2 Domain Models

Each database table has a corresponding C# domain model with data annotations for validation. Key models include:

Model	Key Properties
Booking	BookingID, GuestID, RoomID, CheckIn, CheckOut, Status, TotalAmount
Guest	GuestID, FullName, CNIC, Email, Phone, Address, RegisteredAt
Room	RoomID, RoomNumber, TypeID, Floor, PricePerNight, Status
Payment	PaymentID, BookingID, Amount, PaymentDate, Method, Status
Invoice	InvoiceID, BookingID, TotalAmount, GeneratedDate, PaidStatus
Cancellation	CancellationID, BookingID, Reason, RefundAmount, CancelDate
Housekeeping	TaskID, RoomID, EmployeeID, ScheduledDate, Status, Notes
RoomService	ServiceID, BookingID, ServiceTypeID, Quantity, RequestTime, Status
Login	LoginID, EmployeeID, Username, PasswordHash, IsActive, LastLogin
Employee	EmployeeID, FullName, CNIC, Phone, Role, Salary, HireDate

4.3 Business Logic in Service Layer

The service layer enforces the following automated business rules:

- **Booking Creation:** TotalAmount is automatically calculated as PricePerNight x number of nights. The user cannot manually enter an incorrect amount.
- **Cancellation:** When a cancellation is created, the system automatically sets RefundAmount equal to the booking's TotalAmount and updates the booking Status to Cancelled.
- **Payment:** When a payment is recorded, Amount is set from the booking's TotalAmount. The corresponding invoice is automatically marked as PaidStatus = true.
- **Invoice:** When an invoice is generated, TotalAmount is pulled directly from the booking record.

5. Database Design

5.1 Tables

The database contains 15 tables covering all aspects of hotel operations:

Table	Purpose
RoomType	Defines room categories (Single, Double, Suite, Deluxe, Family)
Room	Individual rooms with floor, price, and availability status
Amenity	Available amenities (WiFi, AC, Jacuzzi, etc.)
RoomAmenity	Many-to-many link between rooms and amenities
Guest	Guest personal details and registration info
Employee	Staff records including role, salary, and hire date
Login	Employee authentication credentials
Booking	Core booking records with check-in/out dates and status
Payment	Payment transactions linked to bookings
Invoice	Invoices generated after checkout
ServiceType	Types of room services offered (Laundry, Spa, etc.)
RoomService	Guest service requests linked to bookings
Housekeeping	Cleaning and maintenance task assignments
Cancellation	Cancellation records with reason and refund amount
ErrorLog	System error logging for audit purposes

5.2 Key Constraints & CHECK Constraints

- Booking: CHECK (CheckOut > CheckIn) prevents invalid date ranges
- Booking: Status must be one of Confirmed, CheckedIn, CheckedOut, Cancelled
- Room: Status must be one of Available, Booked, Maintenance, CheckedIn
- Employee: Role must be one of Admin, Receptionist, Manager, Housekeeper
- Guest: CNIC must be exactly 13 characters
- Payment: Method must be one of Cash, Card, Online, Cheque
- RoomService: Status must be one of Pending, InProgress, Delivered, Cancelled
- Housekeeping: Status must be one of Pending, InProgress, Done

5.3 Views

Seven database views were created to simplify reporting and data retrieval:

View	Description
------	-------------

vw_AvailableRooms	Lists all rooms currently available for booking
vw_ActiveBookings	Shows all confirmed and checked-in bookings with guest and room details
vw_MonthlyRevenue	Aggregates monthly payment totals for revenue reporting
vw_RoomServiceDetails	Joins room service requests with guest, room, and service type details
vw_HousekeepingTasks	Lists housekeeping assignments with employee and room info
vw_GuestBookingHistory	Shows each guest's total bookings and total amount spent
vw_CancellationReport	Lists all cancellations with guest, room, and refund details

5.4 Stored Procedures

Five stored procedures encapsulate critical multi-step operations with transaction support:

Procedure	Description
sp_MakeBooking	Checks room availability, prevents double booking, inserts booking and updates room status
sp_CheckIn	Validates booking status and updates both booking and room to CheckedIn
sp_CheckOut	Updates booking to CheckedOut, room to Available, and auto-generates invoice
sp_CancelBooking	Cancels confirmed bookings, frees the room, and creates cancellation record
sp_RecordPayment	Inserts payment record and automatically marks the invoice as paid

5.5 Triggers

Three database-level triggers enforce data integrity automatically:

Trigger	Description
trg_AfterBookingUpdate	Automatically sets room status to Available when a booking is cancelled
trg_PreventDoubleBooking	Blocks any INSERT on Booking that would create an overlapping reservation for the same room
trg_LogPaymentErrors	Logs an error entry if a payment with Amount <= 0 is inserted

5.6 Transactions

Three explicit transactions were implemented to demonstrate atomicity:

- Transaction 1 — Room Transfer: Moves a guest from one room to another by updating both room statuses and the booking record atomically.
- Transaction 2 — Refund Processing: Updates payment status to Refunded and marks the invoice as unpaid when a cancellation refund is processed.

- Transaction 3 — Bulk Housekeeping Assignment: Assigns a housekeeping employee to all available rooms that do not already have a task scheduled for today.

6. Real-World Application

The Hotel Booking Management System closely mirrors the software used in real hospitality businesses. The following real-world scenarios are directly supported by the system:

6.1 Front Desk Operations

Receptionists can register new guests, check room availability by date range, create bookings with auto-calculated pricing, and manage check-in and check-out processes. The system prevents double bookings at both the application and database level.

6.2 Financial Management

Every booking generates a corresponding invoice upon checkout. Payments are linked directly to bookings, and the system automatically reconciles payment amounts with booking totals. Refunds are tracked through the cancellation and payment modules.

6.3 Housekeeping Coordination

Housekeeping tasks are assigned to specific employees for specific rooms and dates. Managers can monitor task status (Pending, InProgress, Done) and schedule bulk assignments for all available rooms in a single transaction.

6.4 Guest Services

Guests can request room services such as laundry, meals, airport pickup, and spa services through the RoomService module. Each request is tracked with quantity, time, and delivery status.

6.5 Reporting & Analytics

The seven database views provide instant reporting on available rooms, active bookings, monthly revenue, guest spending history, housekeeping workload, and cancellation trends — all without additional queries.

7. Conclusion

The Hotel Booking Management System successfully demonstrates the practical application of database systems concepts in a real-world business context. The project integrates a fully normalized 15-table MySQL schema with a well-structured ASP.NET Core MVC application to deliver a functional hotel management solution.

Key achievements of this project include:

- A robust relational database schema with proper normalization, foreign keys, CHECK constraints, and UNIQUE constraints
- Seven reporting views, five stored procedures, three triggers, and three multi-step transactions
- A layered application architecture with clear separation between data access, business logic, and presentation
- Automated business rule enforcement in the service layer — including dynamic pricing, refund calculation, and invoice synchronization
- A double-booking prevention mechanism enforced at both the trigger level in MySQL and the repository level in C#

This project provided hands-on experience in database design, SQL programming, and full-stack web development, reinforcing the theoretical concepts taught in CMPE-232L. The system is extensible and can be further enhanced with features such as online booking portals, email notifications, and advanced analytics dashboards.